



ERP Electronics Store

Developer & Technical Documentation

Version 1.0 | July 2026

Software Developers · System Administrators · Superadmins

Table of Contents

1. Document Control
2. System Overview & Architecture
3. Technology Stack
4. Brand Identity & Visual Guidelines
5. System Roles & Permissions
6. Database Schema & Models
7. Authentication & Authorization
8. API Reference
9. Superadmin Module
10. Security Measures
11. Analytics & AI Insights
12. Documentation & Diagrams
13. Development Commands & Environment
14. Deployment
15. Full Business Cycle

1. Document Control

Version	1.0
Date	July 2026
Status	Released
Audience	Software developers, system administrators, superadmins, and technical stakeholders
Scope	Architecture, brand identity, technical design, security, deployment, and administration of the ERP Electronics Store platform
Related documents	User_Manual_EN.pdf, User_Manual_SW.pdf, ERD.drawio, ClassDiagram.drawio, UseCase.drawio, SequenceDiagrams.drawio

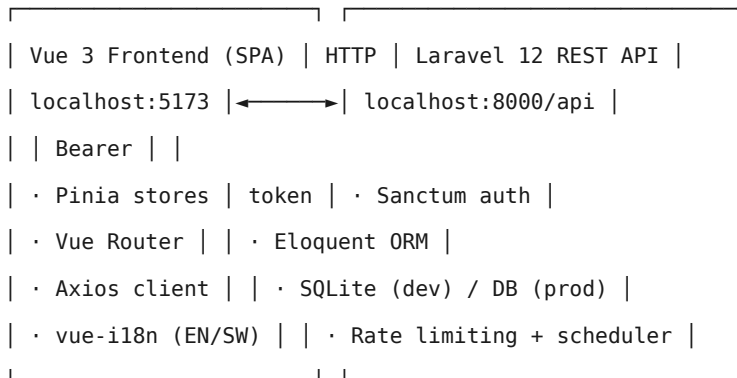
Repositories

Frontend	erp-electronics/ — Vue 3 SPA (storefront + dashboards + superadmin panel)
Backend	erp-electronics-api/ — Laravel 12 REST API
Documentation	docs/ in the frontend repository (manuals, diagrams, this document)

2. System Overview & Architecture

ERP Electronics Store is a multi-tenant SaaS platform for electronics retail businesses. A platform superadmin registers business owners; each owner runs a white-label online store with their own branding, subscription plan, and resource limits. Owners manage employees, who operate the store — processing orders, managing customers, inventory, and support. Customers shop and pay via mobile money. Suppliers fulfil purchase orders through a dedicated portal.

High-Level Architecture



Request Flow

- The SPA calls the API through an Axios client configured with a base URL (VITE_API_URL).
- Authenticated requests send Authorization: Bearer <token> (Sanctum personal access token).
- A 401 response clears the stored token and redirects to the login page.
- API responses are JSON; all /api/* routes render JSON errors automatically.
- Public storefront routes (products, categories, branding) require no authentication.

Cross-Origin Resource Sharing (CORS)

The API permits requests from any origin (allowed_origins: *), which allows the storefront SPA — running on Vite during development or Netlify in production — to consume the API without CORS configuration changes. Preflight requests are handled automatically.

3. Technology Stack

Frontend

Package	Version	Purpose
vue	^3.5	UI framework (Composition API)
vue-router	^5.1	Client-side routing with role guards
pinia	^3.0	State management (auth, session, cart)
pinia-plugin-persistedstate	^4.7	Persist auth token to localStorage
axios	^1.18	HTTP client with interceptors
vue-i18n	^10.0	Internationalization (Swahili / English)
@fortawesome/fontawesome-free	^7.3	Icons (fas fa-*)
chart.js + vue-chartjs	^4.x	Analytics and sales charts
vite	^8.0	Build tooling and dev server
vitest / @vue/test-utils	^4.1 / ^2.4	Unit testing
playwright	^1.61	End-to-end testing

Node requirement: ^22.18.0 || >= 24.12.0

Backend

Package	Purpose
Laravel 12 / PHP 8.4	Application framework
Laravel Sanctum	Token-based API authentication
Eloquent ORM	Database abstraction and relationships
SQLite (development)	Local database (database/database.sqlite)
Filesystem disk	File storage — local in dev, S3 on Laravel Cloud
Gemini API	AI-powered analytics suggestions
Scheduler	Artisan scheduled tasks (daily reports, unpaid-order cleanup, superadmin password reset)

Documentation Tooling

- Python 3 + ReportLab — generates the PDF user manuals and this developer documentation.
- DejaVu Sans fonts — full Swahili character and symbol support in generated PDFs.

- [diagrams.net \(drawio\)](#) — ERD, class, use case, and sequence diagrams; also exported to PDF.

4. Brand Identity & Visual Guidelines

This section defines the official brand identity used across the storefront, dashboards, PDF manuals, and developer material. When producing any asset, use these colors, fonts, and the logo exactly as specified.

4.1 Logo

The logo is a red rounded-corner square with a white lightning bolt, representing electrical energy and fast service. The primary logo is defined in `public/favicon.svg` (64 × 64 viewBox):



Primary logo
mark

Rule	Guideline
Clear space	Maintain padding of at least the bolt height on all sides.
Minimum size	Never render below 16 × 16 px on screen.
Backgrounds	Preferred on white or on the brand red square itself.
Do not	Recolor the bolt, rotate the mark, add effects, or place it on low-contrast backgrounds.
File source	<code>public/favicon.svg</code> (frontend repo).

4.2 Brand Color Palette

The palette below is used consistently across the UI and all documentation. Hex values are authoritative.

#e74c3c	Brand Red (primary)	Buttons, CTAs, active tabs, price highlights, icons, selection color, cover accents
#2c3e50	Dark Slate (secondary)	Navigation, footer, headings, cover background, table headers
	White	Page background, text on brand red
#2980b9	Blue	Informational accents, links in documents

#27ae60	Green	Success states (valid fields, paid/confirmed status)
#888888	Gray	Muted/secondary text, notes
#f5f5f5	Light Gray	Table striping, subtle section backgrounds
#dddddd	Border Gray	Grid lines, hairline dividers

4.3 White-Label Branding (per owner)

Every owner can override the two core brand colors through the superadmin Branding panel (stored on `owner_profiles`):

<code>brand_color</code>	Primary accent color — default #e74c3c
<code>brand_color_secondary</code>	Secondary color — default #2c3e50
<code>brand_store_name</code>	White-label store name shown in the storefront
<code>brand_tagline</code>	Store tagline shown in the storefront

4.4 Typography

Context	Typeface	Notes
Web UI (body)	Inter (weights 300-800)	Loaded from Google Fonts; fallback -apple-system, BlinkMacSystemFont, sans-serif
Web UI (headings)	Inter — SemiBold/Bold	Body text 14 px; page titles 28 px
PDF documents	DejaVu Sans	Full Latin + Swahili + symbol coverage
Code / API	DejaVu Sans Mono	Code blocks and terminal output in this document

5. System Roles & Permissions

Role	Description	Entry point
Superadmin	Platform administrator. Registers owners, manages subscriptions, limits, branding, and owner passwords.	/superadmin
Owner	Business owner. Runs the store: products, employees, orders, inventory, accounting, reports, commissions.	/owner
Employee	Store staff. Processes orders, manages customers, updates delivery, answers support, views earnings.	/employee
Customer	End shopper. Browses the catalog, checks out, tracks orders, contacts support.	/customer
Supplier	External supplier. Views and fulfils the owner's purchase orders.	/supplier

Permissions Matrix

Capability	Customer	Employee	Owner	Superadmin	Supplier
Browse products / shop	✓	✓	✓	✓	✗
Place and view own orders	✓	✓	✓	✗	✗
Process orders (all)	✗	✓	✓	✗	✗
Manage customers	✗	✓	✓	✗	✗
Manage employees	✗	✗	✓	✗	✗
Manage products / branches / shipping	✗	✗	✓	✗	✗
Accounting & financial reports	✗	✗	✓	✗	✗
Commissions & inventory	✗	✓ (earnings only)	✓	✗	✗
Manage owners (CRUD)	✗	✗	✗	✓	✗
Subscriptions, limits, branding	✗	✗	✗	✓	✗
Reset owner passwords / unlock	✗	✗	✗	✓	✗
Supplier portal (view / update POs)	✗	✗	✗	✗	✓

6. Database Schema & Models

The database consists of 28 tables. A full entity-relationship diagram is available as ERD.drawio (crow's foot notation, 35+ relationships).

Tables by Domain

Domain	Tables
Identity	users, customer_profiles, employee_profiles, owner_profiles
Catalog	categories, products, product_variants, inventory
Commerce	addresses, branches, orders, order_items, payments, payment_providers, shipping_rules
Support	support_messages, conversations, conversation_messages
Accounting	accounts, journal_entries, journal_lines, accounting_reports
Operations	commissions, inventory_transactions, purchase_orders, purchase_order_items, suppliers, stock_alerts
Notifications	notifications

Key Models & Relationships

- User (polymorphic) — one customer_profile, employee_profile, or owner_profile; role gates access via middleware.
- employee_profile — branch_id, position, department, commission_rate, hire_date, employee_code; has many employee_guarantors (Wadhamini) and many employee_documents (contracts, background checks).
- Product — belongs to a category; has many product_variants and inventory records.
- Order — belongs to a user and optional branch; has many order_items, a payment, and a delivery/tracking payload.
- Accounting — accounts form a chart of accounts; journal_entries contain journal_lines (double-entry). Payments and cancellations create journals automatically.
- Commissions — created per employee per paid order; paid individually or in bulk by the owner.
- Purchase orders — link suppliers and purchase_order_items; receiving stock creates inventory transactions and journals.
- Conversations — owner ↔ superadmin and owner ↔ customer messaging with real-time unread badges.

7. Authentication & Authorization

Token Flow

1. POST /api/auth/login → { email, password }
2. Backend validates credentials and returns:
{ token, user, must_change_password, superadmin_password_expired }
3. Frontend stores the token in localStorage (key: auth_token)
4. Every request sends: Authorization: Bearer <token>
5. A 401 response clears the token and redirects to /login

Password Policy

- Minimum 8 characters with at least one uppercase, one lowercase, one number, and one special character.
- Enforced on both the frontend (real-time rules) and the backend (custom validation messages).
- Password expiry: password_changed_at must be newer than 3 days, otherwise a forced change modal appears on login.

Default Passwords

Employee	Full name in capitals — e.g. "MATHEW ZACHARIA"; owner can reset anytime
Owner (created by superadmin)	Full name in capitals — e.g. "JOHN DOE"; forced change on first login
Superadmin	SuperAdmin@2026 — auto-reset every 6 months via scheduled artisan command

Account Lockout

After 5 failed login attempts the account locks for 30 minutes (users.locked_until). The API returns HTTP 423 with a remaining-minutes message; the login form also reports remaining attempts.

8. API Reference

All endpoints are JSON and mounted under the /api prefix. Base URLs — development: `http://localhost:8000/api`; production: `https://erp-electronics-api-production-ix4tl.laravel.cloud/api`.

Public Endpoints

Method	Route	Notes
POST	/auth/register	Customer signup (throttled)
POST	/auth/login	Login (throttled)
GET	/products	List products
GET	/products/featured	Featured products
GET	/products/{slug}	Product detail
GET	/categories	List categories
GET	/categories/{slug}	Category with products
GET	/payment-providers	Enabled providers
POST	/payments/webhook	Payment webhook
POST	/shipping/calculate	Shipping cost estimate
GET	/settings/payment	Public payment settings
GET	/settings/branding	Active owner branding

Authenticated Endpoints (Bearer token)

Group	Key routes	Access
Auth	/auth/logout, /auth/profile (GET/PUT), /auth/change-password	any
Employees	/employees (GET/POST/PUT), /employees/{user}/toggle-status, /assign-branch, /reset-password, /documents (CRUD + download)	owner
Branches	/branches CRUD + set-default	owner
Customers	/customers (GET/PATCH/DELETE)	employee, owner
Products (manage)	/products-manage, /products CRUD	owner
Cart / Orders	/cart, /orders, /orders/{id}/status, /delivery, /return	any / employee, owner

Group	Key routes	Access
Accounting	/accounts, /journal-entries, /reports/trial-balance, /profit-loss, /balance-sheet, /general-ledger	owner
Reports	/reports/daily, /reports/summary	employee, owner
Commissions	/commissions, /commissions/summary, /commissions/{id}/pay, /pay-all, /my-earnings	owner / employee
Inventory	/inventory (index, adjust, transactions, low-stock, dashboard)	owner
Purchase orders	/purchase-orders CRUD + receive	owner
Suppliers	/suppliers CRUD + portal	owner / supplier
Superadmin	/superadmin/stats, /owners CRUD, /owners/{id}/branding, /passwords/status, /owners/{id}/reset-password, /unlock-account	superadmin

Rate Limits

Limiter	Limit	Keyed by
api (global)	120 requests per minute	Authenticated user ID, else IP
login	5 requests per minute	IP address
register	3 per minute and 10 per day	IP address

Exceeded limits return HTTP 429 with X-RateLimit-Remaining / X-RateLimit-Reset headers.

9. Superadmin Module

The superadmin is the platform administrator responsible for onboarding and managing business owners. Access the panel at /superadmin (role-guarded). This module is fully documented here for administrators and developers; it is intentionally not described in the end-user manuals.

9.1 System Overview

- System Statistics — total customers, total orders, total revenue, and active owners at a glance.
- Owners Table — all registered owners with company name, plan, status, and registration date.

9.2 Managing Owners

- Create Owner — registers an owner with name, email, phone, and company name. Default password is the full name in capitals; the response exposes `default_password` to share securely.
- Toggle Active/Inactive — enable or disable an owner's account instantly.
- Delete Owner — removes the owner and all associated data.

9.3 Owner Details

Opening an owner reveals:

- Subscription — plan (Starter, Professional, Enterprise), status, and expiry date.
- Limits — maximum products and employees the owner may register.
- Branding — white-label configuration: store name, tagline, logo upload, and the two brand colors (see §4.3).

9.4 Owner Password Management

- Reset Password — resets to the default (full name in capitals) and forces a change on next login.
- Set Password — assigns an explicit password.
- Force Password Change — clears `password_changed_at` so the owner must change it.
- Unlock Account — clears `locked_until` after a 30-minute lockout.
- Passwords Status — overview of every owner's password state (changed / needs change / expired).

9.5 Superadmin Inbox

The superadmin communicates with owners through the Inbox (owner ↔ superadmin conversations). Conversations appear in real time with unread badges.

9.6 Superadmin Password Lifecycle

The default superadmin password is SuperAdmin@2026. A scheduled command (`php artisan superadmin:reset-password`) auto-resets it every 6 months; login and profile responses expose `superadmin_password_expired` so the UI can prompt for a change.

10. Security Measures

10.1 API Rate Limiting

Laravel's named rate limiters protect every API route. The api group limiter is attached to the whole /api/* group; the login and register limiters sit on the public auth routes.

```
// app/Providers/AppServiceProvider.php

RateLimiter::for('api', fn (Request $r) =>
Limit::perMinute(120)->by($r->user()?->id ?: $r->ip()));

RateLimiter::for('login', fn (Request $r) => Limit::perMinute(5)->by($r->ip()));

RateLimiter::for('register', fn (Request $r) => [
Limit::perMinute(3)->by($r->ip()),
Limit::perDay(10)->by($r->ip()),
]);
```

Registered with `$middleware->throttleApi()` in `bootstrap/app.php` and `throttle:login / throttle:register` on the auth routes.

10.2 Session Termination (Idle Timeout)

The frontend session store (`src/stores/session.js`) enforces automatic logout:

- 15 minutes of inactivity → session ends (mouse, keyboard, touch, scroll, and click activity reset the timer).
- At 14 minutes a warning modal shows a 60-second countdown; any activity dismisses it.
- Leaving the dashboard — switching tabs/apps pauses the timer; returning within 10 minutes continues the session, otherwise the user is signed out.
- A `session_last_active` timestamp is persisted, so returning to a closed tab after the idle window forces logout on load.
- Logout clears `auth_token` from `localStorage` and revokes the Sanctum token on the server.

10.3 Passwords & Lockout

- Complexity enforced at the API level (8+ chars; upper, lower, number, symbol).
- Accounts lock after 5 failed attempts for 30 minutes.
- Passwords older than 3 days trigger a mandatory change; superadmin password resets every 6 months.

10.4 File Uploads

- Employee documents accept only PDF, JPG, PNG, DOC, DOCX, up to 20 MB each.
- Files are stored on the configured filesystem disk under `employee-documents/` (local in development; S3 on Laravel Cloud).
- Downloads are served only through the authenticated owner endpoints.

11. Analytics & AI Insights

Sales Analytics

`AnalyticsController::sales()` accepts `?months=12` and computes, in parallel: monthly sales, monthly items sold, monthly profit, and monthly cancellations. It returns a gap-free month list plus `category_breakdown`, `top_products`, and a summary (total revenue, profit, orders, items sold, average order value, profit margin, revenue and order growth).

AI Suggestions

The owner dashboard can call `POST /analytics/ai-suggestions` with the sales payload. The backend builds a Tanzania-specific business prompt and calls Gemini 2.0 Flash (temperature 0.7, max 2048 tokens, configured via `GEMINI_API_KEY`). If the AI response is unavailable, rule-based fallback suggestions are returned.

Suggestions are bilingual (`title_sw/title_en`, `description_sw/description_en`) with a priority (high/medium/low), category (inventory/pricing/marketing/growth/operations), expected impact, and a source of "ai" or "fallback".

12. Documentation & Diagrams

PDF Documents

File	Audience	Notes
User_Manual_EN.pdf	End users	English user manual, 24 chapters (v3.0)
User_Manual_SW.pdf	End users	Swahili user manual, 24 chapters (v3.0)
Developer_Documentation.pdf	Developers & admins	This document (v1.0)

System Diagrams (docs/)

Diagram	Contents
ERD.drawio (+ PDF)	28 tables, 35+ relationships, crow's foot notation
ClassDiagram.drawio (+ PDF)	28 model classes with attributes, methods, and relationships
UseCase.drawio (+ PDF)	6 actors, 45+ use cases, include/extend relationships
SequenceDiagrams.drawio (+ PDF)	7 flows: customer checkout, order processing, branch + employee management, accounting, commission, inventory, purchase orders

Generator Scripts

- `generate_manual.py` — user manuals (EN + SW) using ReportLab.
- `generate_dev_doc.py` — this developer documentation.

13. Development Commands & Environment

Frontend (erp-electronics/)

Command	Purpose
npm run dev	Start Vite dev server (port 5173)
npm run build	Production build
npm run lint	Run oxlint + eslint
npm run format	Format with oxfmt
npm run test:unit	Run Vitest unit tests
npm run test:e2e	Run Playwright end-to-end tests

Backend (erp-electronics-api/)

Command	Purpose
php artisan serve	Start dev server (port 8000)
php artisan migrate	Run migrations
php artisan migrate:fresh --seed	Reset database and re-seed
php artisan report:daily	Generate daily report
php artisan orders:cleanup-unpaid	Clean up unpaid orders
php artisan superadmin:reset-password	Reset superadmin password if 6 months elapsed
php artisan schedule:work	Run the scheduler
php artisan route:list	List routes
php artisan test	Run the test suite

Environment Variables

```
# Frontend (.env)
VITE_API_URL=http://localhost:8000/api

# Backend (.env)
DB_CONNECTION=sqlite
DB_DATABASE=/absolute/path/to/database.sqlite
GEMINI_API_KEY=your-gemini-api-key
FILESYSTEM_DISK=local # "s3" on Laravel Cloud
```

`AWS_ACCESS_KEY_ID= # when using S3`

`AWS_SECRET_ACCESS_KEY=`

`AWS_DEFAULT_REGION=us-east-1`

`AWS_BUCKET=`

`AWS_USE_PATH_STYLE_ENDPOINT=false`

14. Deployment

14.1 Backend — Laravel Cloud

- Pushing to the main branch triggers an automatic deployment.
- Production environment id: env-a2578b2b-6a5c-4d19-9c2e-d927b4edafde; app id: app-a2578b29-e3ec-4754-a717-a6a208f4f512.
- Deployment verification: `cloud command:run <env-id> --cmd="php artisan test" -n`.
- Health check: GET /up on the production domain.

14.2 Frontend — Netlify

- Set `VITE_API_URL=https://erp-electronics-api-production-ix4tl.laravel.cloud/api` as a build-time environment variable.
- Build command `npm run build`; publish directory `dist`; enable SPA redirects to `index.html`.
- The API accepts the Netlify origin via CORS (`allowed_origins: *`).

Rate-limit keys are IP-based, so an entire office sharing one public IP shares the 5/min login budget. This is intentional to block brute-force traffic.

15. Full Business Cycle

Customer Journey

Browse → Add to cart (variant, quantity) → Checkout (delivery + payment)
→ Order created (pending_payment)
→ Paid (cash/ClickPesa auto; mobile money confirmed by employee)
→ Auto-journal (Revenue + COGS), inventory decremented, commission created
→ Processing → Shipped (tracking) → Delivered
→ Cancel at any stage: reversal journals, stock restored, commission deleted

Employee Flow

- Login → dashboard with stats and quick actions.
- Confirm payment (type name) → order to paid → process → ship → deliver.
- View earnings at /employee/earnings; handle stock alerts and support.

Owner Flow

- Dashboard → products → employees (guarantors + documents + commission rate) → orders.
- Inventory, suppliers, purchase orders, commissions, accounting, stock alerts, branches.
- Reports & analytics with AI suggestions; owner inbox for customer and administrator conversations.

Auto-Journal Chain

Customer pays TSh 500,000
→ DR Accounts Receivable 500,000 / CR Sales Revenue 500,000
→ DR Cost of Goods Sold 300,000 / CR Inventory 300,000
→ inventory transactions, stock alerts, commission = profit × rate
Order cancelled
→ DR Sales Revenue 500,000 / CR Accounts Receivable 500,000
→ DR Inventory 300,000 / CR Cost of Goods Sold 300,000
Owner pays commission
→ DR Commission Expense / CR Cash